Lab 01 — TCP Three-Way Handshake & Flag Analysis

Protocol: TCP (Transmission Control Protocol)
Tool: Wireshark on Kali Linux
Difficulty: Beginner → Foundation for all other labs
Time Taken: ~2 hours

Objective

Capture and analyze a complete TCP connection lifecycle — from the initial handshake to graceful termination — and identify every flag's role at the packet level. Understanding TCP flags is foundational for detecting port scans, SYN floods, session hijacking, and RST injection attacks.

Protocol Background — Why TCP Matters

TCP (Transmission Control Protocol) is the backbone of reliable internet communication. Unlike UDP, TCP guarantees delivery through a connection-oriented model. Every web page load, SSH session, HTTP request, and database query uses TCP.

TCP operates on a flag-based state machine. Each packet carries one or more flags in the TCP header that tell the receiving end what to do with it:

Flag	Bit	Full Name	Purpose
SYN	0x02	Synchronize	Initiate connection, exchange sequence numbers
ACK	0x10	Acknowledge	Confirm receipt of data
FIN	0x01	Finish	Gracefully close connection
RST	0x04	Reset	Abruptly terminate connection
PSH	0x08	Push	Send buffered data immediately
URG	0x20	Urgent	Prioritize this data

What I Captured

The TCP Three-Way Handshake

The connection between my Kali machine and Ubuntu target goes through a precise 3-step sequence before any data is exchanged:

```
Client (Kali)          Server (Ubuntu)

   |                    |

   |—— SYN ————————————▶ |  Step 1: "I want to connect, my seq=X"

   |                    |

   | ◀—— SYN-ACK ————————| Step 2: "OK, my seq=Y, I got your X"

   |                    |

   |—— ACK ————————————▶ |  Step 3: "Got it, we're connected"

   |                    |

   |═══════ DATA EXCHANGE ═══════|

   |                    |

   |—— FIN-ACK ————————▶ |  Teardown begins

   | ◀—— FIN-ACK ————————|

   |—— ACK ————————————▶ |  Connection closed
```

Wireshark Filters Used

```
# Capture all TCP traffic

tcp


# Isolate the handshake — SYN packets only

tcp.flags.syn == 1 && tcp.flags.ack == 0


# SYN-ACK responses

tcp.flags.syn == 1 && tcp.flags.ack == 1


# Connection teardown

tcp.flags.fin == 1
```

Abrupt resets

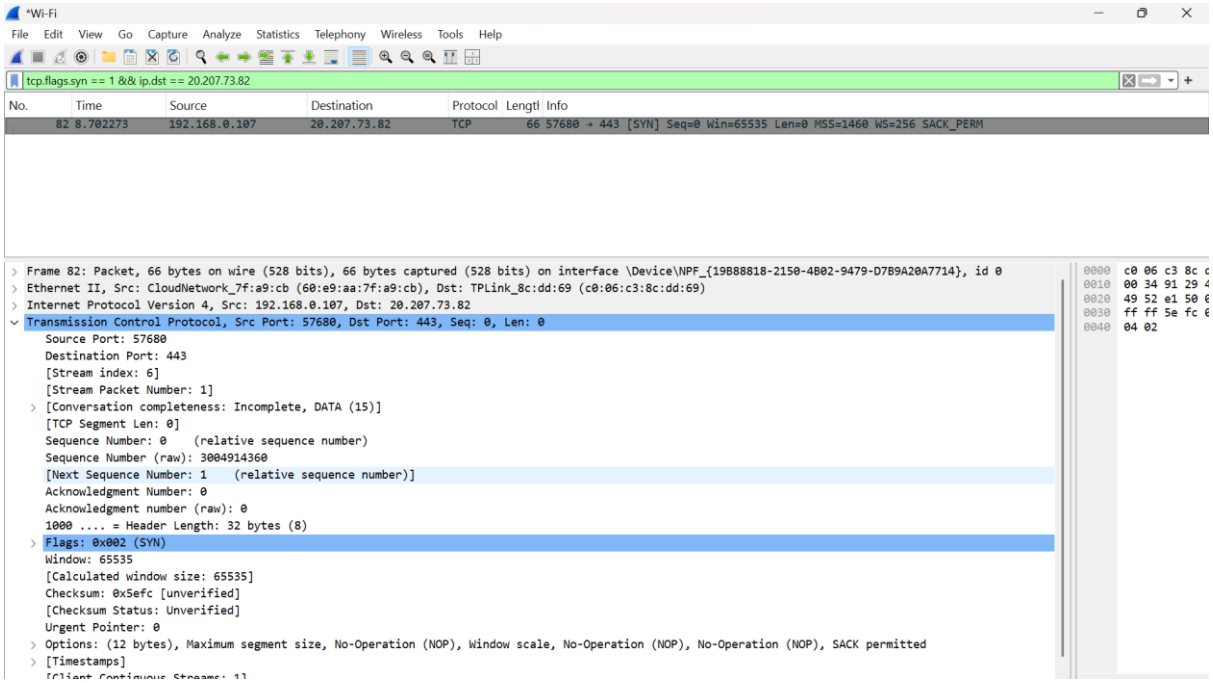
tcp.flags.reset == 1

Filter by specific host pair

ip.addr == 192.168.x.x && tcp

Screenshots & Findings

Screenshot 1 — SYN Flag (Connection Initiation)



What this shows:

- Client sends a TCP packet with only the **SYN flag set**
- Sequence number is a random initial value (ISN — Initial Sequence Number)
- Wireshark shows Flags: 0x002 (SYN) in the TCP header breakdown
- No data payload — this is purely a handshake packet

Key fields visible in Wireshark:

Source Port: [random ephemeral port, e.g., 54321]

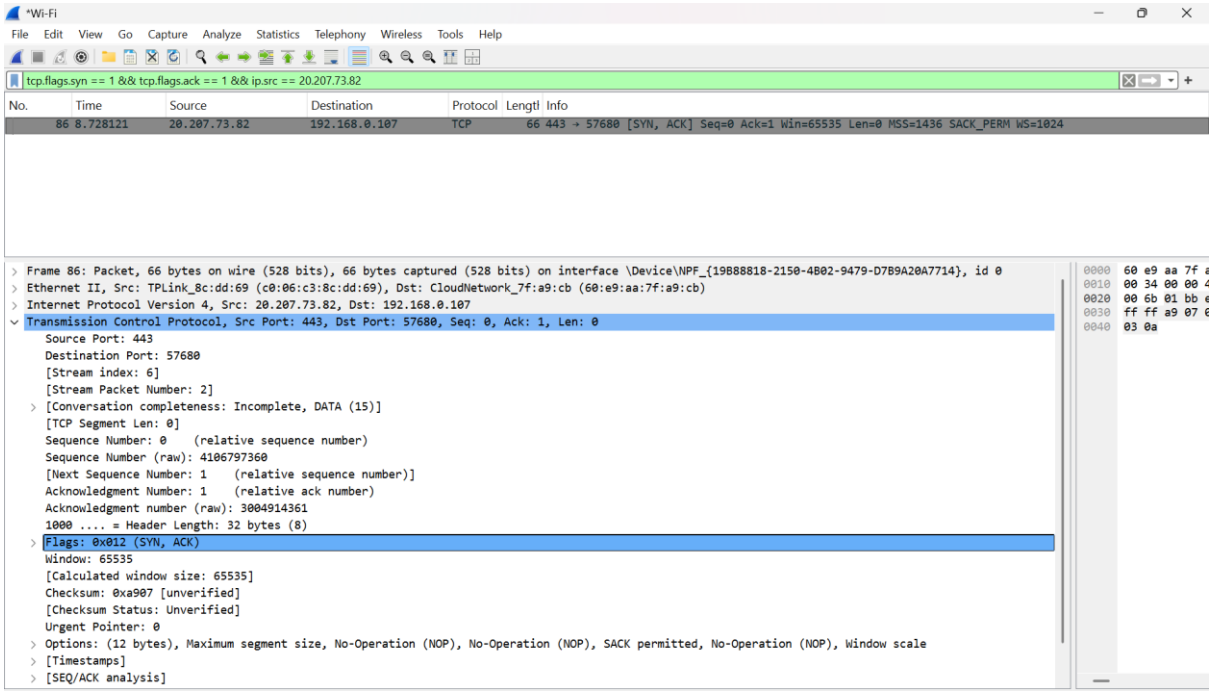
Destination Port: 80 (HTTP) or 22 (SSH)

Sequence Number: [ISN - randomly generated]

Flags: 0x002 → SYN = 1, all others = 0

Window Size: 65535 (advertised receive buffer)

Screenshot 2 — SYN-ACK Flag (Server Response)



What this shows:

- Server responds with **both SYN and ACK flags set**
- Server's own ISN (Initial Sequence Number) is set here
- ACK number = Client's ISN + 1 (confirms receipt)
- This is the only packet where a server sends SYN + ACK together

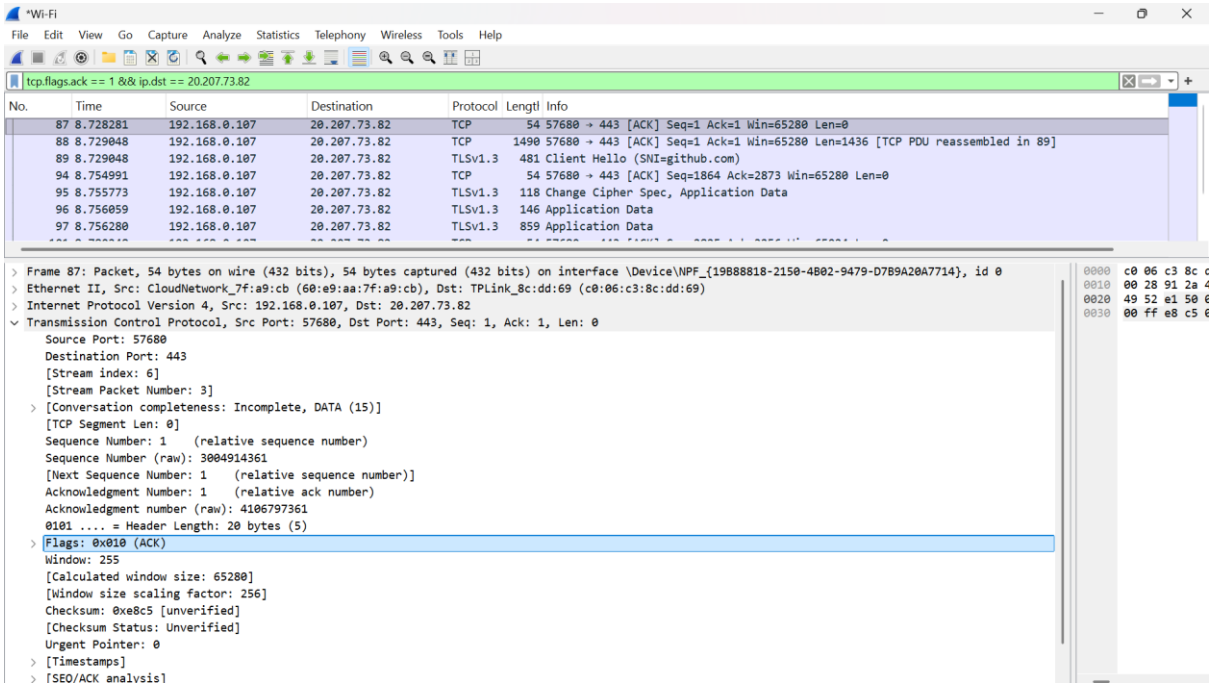
Key fields visible:

Flags: 0x012 → SYN = 1, ACK = 1

Sequence Number: [Server's ISN]

Acknowledgment: [Client ISN + 1]

Screenshot 3 — ACK Flag (Handshake Complete)

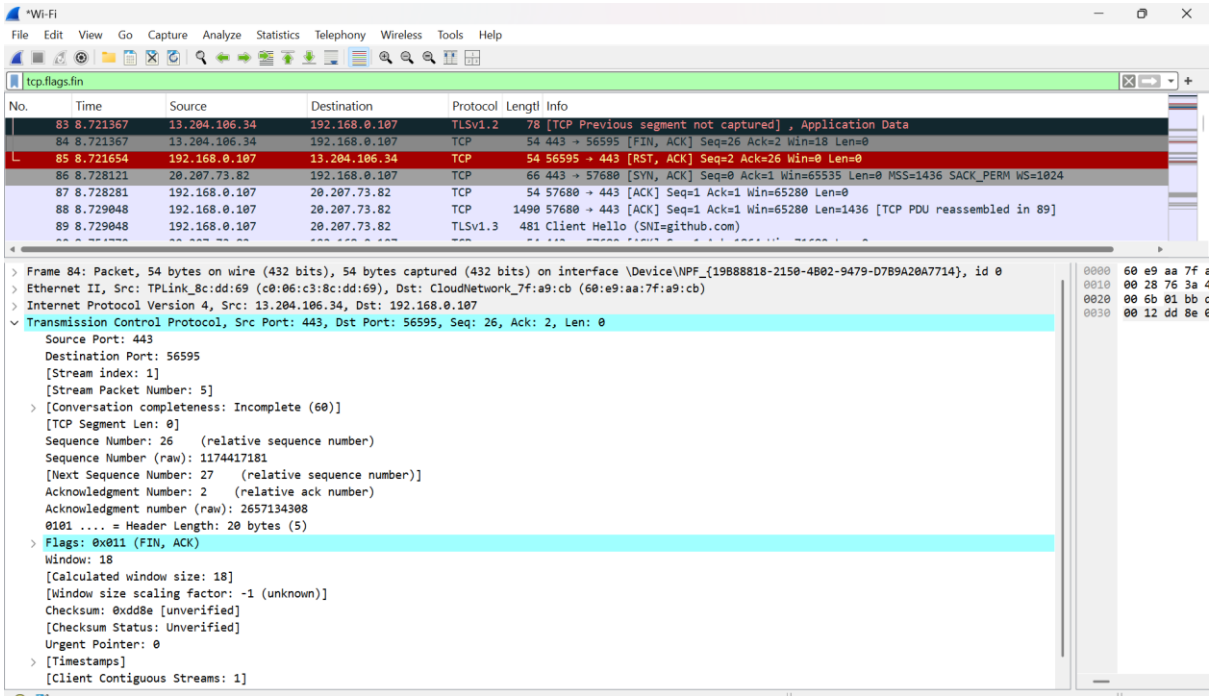


What this shows:

- Client sends final ACK to complete the three-way handshake
- At this point **the connection is established** — data can flow
- ACK number = Server's ISN + 1

Key observation: After this packet, you'll see the actual data packets (HTTP GET, SSH banner, etc.) flowing in the capture.

Screenshot 4 — FIN-ACK Flag (Graceful Connection Close)

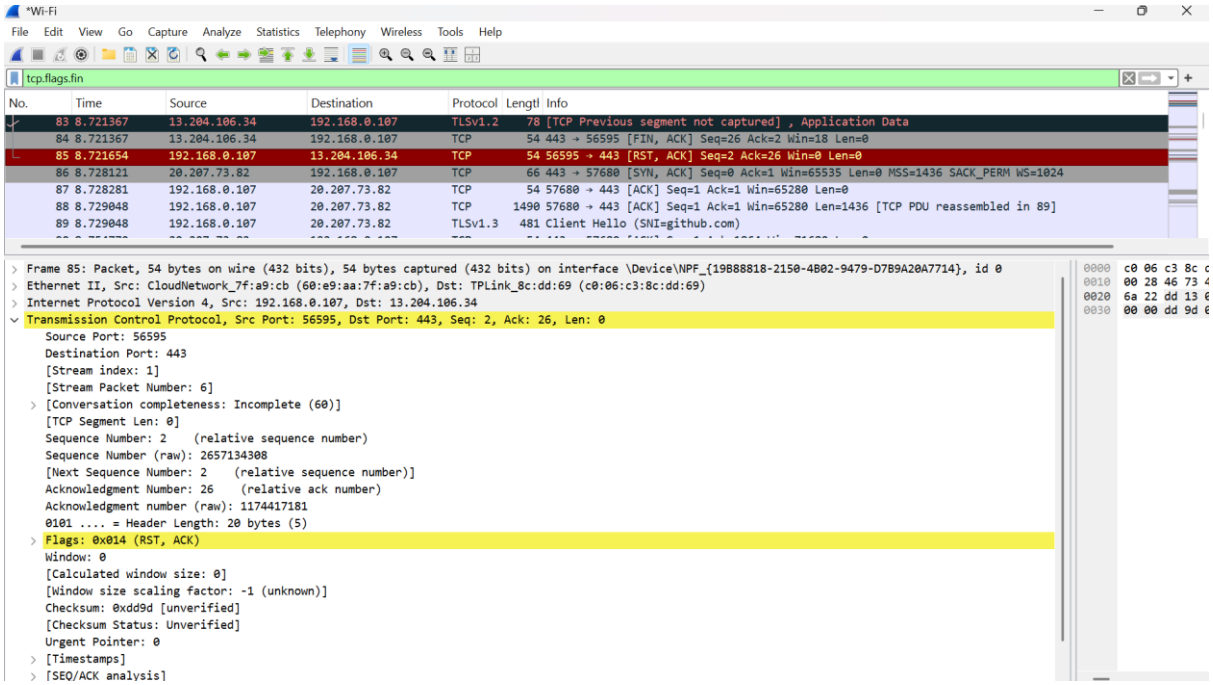


What this shows:

- Either side (client or server) initiates a **graceful shutdown** with FIN
- The full four-way teardown: FIN → ACK → FIN → ACK
- No data is lost — both sides agree to close

Why this matters: A graceful close means the application finished properly. An **abrupt RST** (below) means something went wrong — or an attacker interfered.

Screenshot 5 — RST Flag (Connection Reset)



What this shows:

- RST (Reset) flag abruptly terminates a connection
- **No graceful teardown** — connection dies immediately
- The receiving end must discard any pending data

When RSTs appear in the wild:

- Port is closed (firewall or no listener)
- **RST injection attack** — attacker forges a RST to kill someone's TCP session
- TCP session timeout
- Nmap SYN scan detection (server RSTs unanswered SYNs)

Sequence Number Analysis

One of the most important TCP concepts visible in Wireshark is **sequence number tracking**:

Packet 1 (SYN): Seq=1000, Ack=0

Packet 2 (SYN-ACK): Seq=5000, Ack=1001

Packet 3 (ACK): Seq=1001, Ack=5001

Packet 4 (DATA): Seq=1001, Ack=5001, Len=512

Packet 5 (ACK): Seq=5001, Ack=1513

Wireshark's **"Relative Sequence Numbers"** feature (enabled by default) resets ISNs to 0 for readability. Disable this in Preferences to see raw sequence numbers — critical for detecting **sequence number prediction attacks**.

Security Implications — What an Attacker Can Do With This

1. SYN Flood (DoS Attack)

An attacker sends thousands of SYN packets **without completing the handshake**. The server allocates memory for each half-open connection (in the SYN_RECEIVED state), eventually exhausting its connection table.

What it looks like in Wireshark:

Thousands of SYN packets from one/many IPs, no corresponding ACKs

tcp.flags.syn == 1 && tcp.flags.ack == 0

Detection: SYN rate far exceeds ACK rate. SYN cookies are the primary defense.

2. TCP Session Hijacking

If an attacker can **predict sequence numbers**, they can inject forged packets into an established TCP session — effectively hijacking it mid-stream.

Legitimate Client —— TCP Session ——▶ Server

Attacker ———————— Forged packet with correct Seq# ——▶ Server

Detection: Out-of-order sequence numbers, duplicate ACKs, unexpected RSTs.

3. RST Injection Attack

An attacker sends forged RST packets to **terminate someone else's TCP connection** — used in network censorship (Great Firewall of China) and man-in-the-middle attacks.

Detection filter: RSTs not matching established connection state

tcp.flags.reset == 1

4. Port Scanning via TCP Flags

Nmap uses different flag combinations to map open ports:

- **SYN scan:** Send SYN, if SYN-ACK returns → port open; if RST → port closed
- **FIN scan:** Send FIN to closed port → RST returned; open port → silence

---

Defender Perspective — Detection Rules

Snort/Suricata IDS rules for SYN flood:

alert tcp any any -> $HOME_NET any (flags:S; threshold: type both, track by_src, count 100, seconds 10; msg:"SYN Flood Detected"; sid:1001;)

Firewall rules:

```
# Linux iptables — enable SYN cookies
echo 1 > /proc/sys/net/ipv4/tcp_syncookies

# Rate limit new connections
iptables -A INPUT -p tcp --syn -m limit --limit 1/s -j ACCEPT
```

Key Takeaways

1. **Every TCP connection starts with SYN** — any SYN without a corresponding SYN-ACK is suspicious
2. **RST packets can be forged** — a sudden RST mid-session could be an attack
3. **Sequence numbers prevent replay attacks** but can be predicted in weak implementations
4. **Wireshark's TCP stream reassembly** (Follow → TCP Stream) shows the full application-layer conversation

---